FINSIGHT BANKING ANALYTICS PLATFORM

A Medallion Architecture Implementation on Microsoft Fabric

Bronze → Silver → Gold | PySpark + Dataflow Gen2 | Delta Lake | Power BI

Technical Implementation & Project Documentation Report

Transaction Fraud, Customer, Product & Branch Analytics

Field	Details
Project Name	FinSight Banking Analytics Platform
Architecture Pattern	Medallion Architecture (Bronze → Silver → Gold Lakehouse)
Platform	Microsoft Fabric (Lakehouse, Notebooks, Dataflow Gen2, Data Pipelines)
Core Compute Engine	Apache Spark (PySpark) + Power Query (Dataflow Gen2)
Storage Format	Delta Lake
Serving / BI Layer	SQL Analytics Endpoint + Power BI
Scale	6.3M+ transactions, 50K customers, 200 branches, 10 products
Document Type	Technical Architecture & Implementation Report
Report Date	July 2026

Table of Contents

1. Executive Summary

This report documents the design, build, and deployment of FinSight, an end-to-end banking analytics pipeline on Microsoft Fabric. The platform ingests raw transaction, customer, product, and branch records, validates and cleanses them, and produces a set of curated Gold tables that power executive reporting on transaction volume, fraud incidence, customer segmentation, and branch/product performance.

The solution follows the Medallion Architecture, separating data into Bronze (raw, validated ingestion), Silver (cleansed, conformed, and enriched), and Gold (aggregated, analytics-ready) layers. Transaction data is processed through dedicated PySpark notebooks — _nb_bronze_validation.ipynb, nb_silver_transformation.ipynb, and nb_gold_daily_summary.ipynb — while the customer, product, and branch reference data are cleansed through a Dataflow Gen2 (Power Query) pipeline. The full source code for all three notebooks is reproduced and explained throughout this document.

The platform currently processes over 6.3 million transaction records, 50,000 customer records, 200 branches, and 10 products, exposing them through a SQL Analytics Endpoint and a three-page Power BI report (Executive Summary, Customer Analytics, and Product & Branch Analytics).

Key Outcomes

- A validated, three-stage pipeline (PySpark + Dataflow Gen2) turning raw CSV extracts into governed Delta tables.
- An automated Bronze validation gate that checks row counts, null keys, and duplicate keys before data is trusted downstream.
- Four Gold-layer KPI tables covering transactions, branches, customers, and products.
- A three-page executive Power BI report covering fraud monitoring, customer segmentation, and branch/product performance.

2. Problem Statement

Retail banks generate very large volumes of transactional data — payments, transfers, cash movements — alongside slower-changing reference data about customers, products, and branches. Raw exports of this data arrive as flat CSV files with string-typed columns, inconsistent naming (e.g., oldbalanceOrg, nameDest), and no built-in mechanism to flag suspicious or invalid records before they reach reporting.

Without a structured pipeline, three concrete problems emerge:

- No automated gate to catch broken or incomplete source extracts before they contaminate downstream tables — a partial file load could silently understate transaction volume or hide fraud.
- Raw transaction fields are untyped strings with cryptic banking-system names, making it error-prone for analysts to compute reliable balance or fraud metrics directly.
- No shared, pre-aggregated layer for fraud counts, customer segments, or branch performance — every report re-derives these from millions of raw rows.

The goal of this project is to build a governed pipeline — with an explicit validation gate, typed and enriched Silver tables, and a small set of trustworthy Gold aggregates — that removes this repeated effort and gives bank operations and risk teams a consistent foundation for decision-making.

3. Business Objectives & Goals

By implementing this end-to-end analytics platform, bank operations, risk, and branch management teams gain the ability to:

3.1 Monitor Transaction Volume & Fraud

Track total transaction count, transacted amount, and fraud incidence, broken down by transaction type (TRANSFER, CASH_OUT, CASH_IN, PAYMENT, DEBIT) and by month/year.

3.2 Understand the Customer Base

Segment customers by tier (Standard, Basic, Student, Premium, Business) and age band, and track average income, credit score, and active-vs-inactive status within each segment.

3.3 Manage Branch & Product Performance

Track branch counts and staffing levels by region, active-vs-inactive branch status, and product-level fee and interest-rate ranges by category.

3.4 Establish a Governed, Auditable Data Foundation

Ensure every KPI shown to leadership is backed by an automated validation step and a traceable, repeatable transformation path from raw file to Gold table.

4. Solution Architecture Overview

FinSight is built on Microsoft Fabric using the Medallion Architecture pattern. Transaction data flows through a pure PySpark path (Bronze → Silver → Gold), while the smaller, slower-changing customer, product, and branch reference tables are cleansed via a Dataflow Gen2 (Power Query) transformation before landing in Silver.

Figure 4.1 — Medallion Architecture: Bronze → Silver → Gold pipeline for FinSight Banking Analytics.

4.1 Layer Responsibilities

Layer	Tables	Responsibility
Bronze	bronze_transactions, bronze_customers, bronze_products, bronze_branches	Raw ingestion from CSV; automated validation (row count, null keys, duplicate keys).

Layer	Tables	Responsibility
Silver	silver_transactions, silver_customers, silver_products, silver_branches	Type casting, renaming, enrichment, data-quality flags (transactions); Power Query cleansing (reference tables).
Gold	gold_transaction_summary, gold_branch_summary, gold_customer_summary, gold_product_summary	Business aggregations ready for BI consumption.

4.2 Technology Stack

- Compute: Apache Spark (PySpark) notebooks for transactions and validation; Dataflow Gen2 (Power Query) for reference data.
- Storage: Delta Lake tables inside a Fabric Lakehouse (lh_finsight, OneLake).
- Orchestration: Fabric Data Pipeline (Copy Data + Notebook + Dataflow activities).
- Serving: SQL Analytics Endpoint (vw_gold_* views) and Power BI (three-page executive report).

5. Fabric Workspace Environment

All FinSight artifacts — notebooks, the data pipeline, the Dataflow Gen2, the Lakehouse, and the Power BI report — live inside the FinSight-Banking-Analytics Fabric workspace.

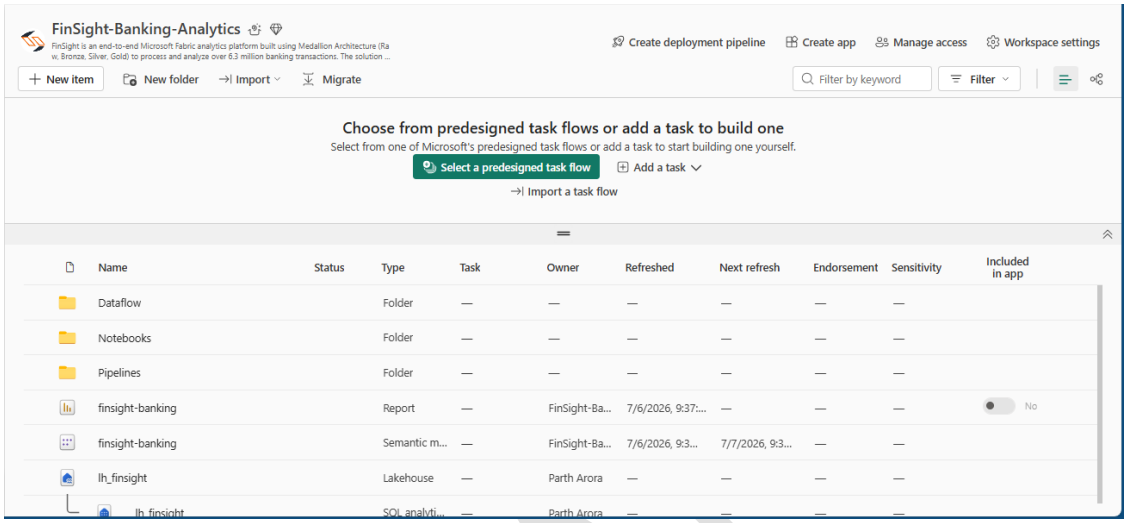

Figure 5.1 — The FinSight Fabric workspace, with the lh_finsight Lakehouse, health_ingest_pipeline / data_ingest_pipeline, bronze/silver/gold notebooks, and the Dataflow transformation all open as tabs.

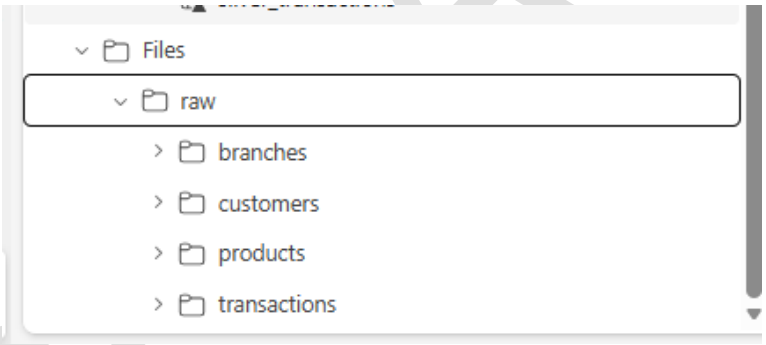

Figure 5.2 — Files/raw landing folder inside lh_finsight, holding the branches, customers, products, and transactions CSV drop zones.

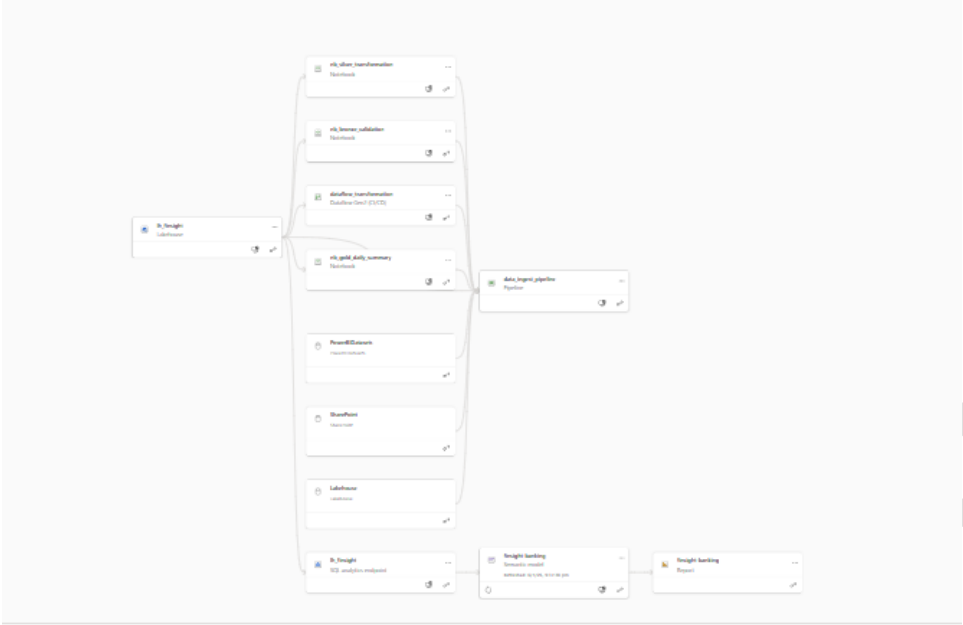

Figure 5.3 — Workspace item list, showing the FinSight-Banking-Analytics Lakehouse item and its dependent notebooks, dataflow, and pipeline.

The three notebooks documented in this report — _nb_bronze_validation, nb_silver_transformation, and nb_gold_daily_summary — are organized alongside the lh_finsight Lakehouse (holding all Bronze, Silver, and Gold tables), a Dataflow Gen2 for reference-data cleansing, and a Data Pipeline that sequences the whole run.

6. Data Sources & Raw Schema

FinSight's raw data consists of four CSV extracts landed under Files/raw/ in the Lakehouse: transactions, customers, products, and branches. The transactions extract is the largest by far, at over 6.3 million rows.

6.1 Raw Transactions Schema

As confirmed by nb_silver_transformation.ipynb's printSchema() output on the Bronze table:

Column	Type (Bronze)	Description
step	string	Sequential time step (1 unit = 1 hour of simulated time)
type	string	Transaction type: TRANSFER, CASH_OUT, CASH_IN, PAYMENT, DEBIT
amount	string	Transaction amount
nameOrig	string	Originating account identifier
oldbalanceOrg	string	Origin account balance before the transaction
newbalanceOrig	string	Origin account balance after the transaction
nameDest	string	Destination account identifier
oldbalanceDest	string	Destination account balance before the transaction
newbalanceDest	string	Destination account balance after the transaction
isFraud	string	Whether the transaction was confirmed fraudulent
isFlaggedFraud	string	Whether the transaction was auto-flagged as suspicious

6.2 Reference Data (Customers, Products, Branches)

- bronze_customers — customer_id, segment, age_band, annual_income, credit_score, is_active, and related demographic fields (~50,000 rows).
- bronze_products — product_id, category, fee, interest_rate (10 rows — one row per banking product).
- bronze_branches — branch_id, region, staff_count, is_active (200 rows).

> *The key columns used for validation differ by table and reflect their grain: nameOrig for transactions (an account can appear many times), customer_id for customers, product_id for products, and branch_id for branches (see Section 7.2).*

7. Bronze Layer — Ingestion & Validation (_nb_bronze_validation.ipynb)

Raw files are landed into four Bronze Delta tables by the Fabric Data Pipeline's Copy Data activities (Section 10). The Bronze layer notebook documented here does not perform the ingestion itself — its role is to validate the landed Bronze tables before Silver processing is allowed to proceed, acting as an automated data-quality gate.

7.1 Parameters & Imports

```
tableName = "table1"
```

_nb_bronze_validation.ipynb — Cell 1: a parameter cell (tagged "parameters"), allowing this notebook to be called with a different table name if invoked as a parameterized pipeline step. In the current run it is unused, as validation instead loops over the tables dictionary below.

```
from pyspark.sql import functions as F
from datetime import datetime
```

_nb_bronze_validation.ipynb — Cell 2: standard imports.

7.2 Validation Configuration

```
tables = {
    'bronze_transactions': {'min_rows': 1000000, 'key_col': 'nameOrig'},
    'bronze_customers':    {'min_rows': 10000,   'key_col': 'customer_id'},
    'bronze_products':     {'min_rows': 5,       'key_col': 'product_id'},
    'bronze_branches':     {'min_rows': 10,      'key_col': 'branch_id'},
}
```

_nb_bronze_validation.ipynb — Cell 3: defines, per Bronze table, the minimum acceptable row count and the key column to check for nulls and duplicates.

7.3 Validation Loop

```
validation_results = []

for table_name, config in tables.items():

    try:

        df = spark.read.format("delta").load(f"Tables/dbo/{table_name}")

        row_count = df.count()

        null_count = (
            df.filter(F.col(config["key_col"]).isNull())
            .count()
        )

        dupe_count = (
            df.groupBy(config["key_col"])
            .count()
            .filter("count > 1")
            .count()
        )
```

```
        status = (
            "PASS"
            if row_count >= config["min_rows"] and null_count == 0
            else "FAIL"
        )

        validation_results.append({
            "table": table_name,
            "row_count": row_count,
            "null_keys": null_count,
            "duplicates": dupe_count,
            "status": status
        })

        print(
            f"{status}: {table_name} | "
            f"Rows={row_count:,} | "
            f"Nulls={null_count} | "
            f"Dupes={dupe_count}"
        )

    except Exception as e:

        print(f"FAIL: {table_name} - Error: {e}")

        validation_results.append({
            "table": table_name,
            "status": "ERROR",
            "error": str(e)
        })
# Save validation log as a Delta table
val_df = spark.createDataFrame(validation_results)

val_df = val_df.withColumn(
    "validated_at",
    F.current_timestamp()
)

val_df.write \
    .format("delta") \
    .mode("append") \
    .saveAsTable("bronze_validation_log")
```

_nb_bronze_validation.ipynb — Cell 4: for each configured table, checks row count against the minimum threshold, counts null keys, counts duplicate keys, assigns a PASS/FAIL status, and appends a timestamped row to the bronze_validation_log audit table.

7.4 Observed Validation Output

Table	Rows	Null Keys	Duplicate Keys	Status
bronze_transactions	6,362,620	0	9,298	PASS
bronze_customers	50,000	0	0	PASS
bronze_products	10	0	0	PASS

Table	Rows	Null Keys	Duplicate Keys	Status
bronze_branches	200	0	0	PASS

Design note: bronze_transactions passes with 9,298 "duplicate" nameOrig values, because the validation logic (row count + zero nulls) does not treat key duplication as a failure condition on its own — duplicates are recorded but not penalized. This is actually correct behavior here: nameOrig is an originating account, and the same account legitimately appears across many separate transactions, so a high duplicate count is expected, not an error. It would, however, be worth renaming key_col in the configuration (e.g., to reference_col) to avoid implying nameOrig should be unique per row.

8. Silver Layer — Cleansing & Enrichment

Silver-layer processing splits across two tools: the high-volume transactions table is transformed with PySpark (nb_silver_transformation.ipynb, detailed below), while the smaller customers, products, and branches reference tables are cleansed using a Dataflow Gen2 (Power Query) pipeline.

8.1 Load Bronze & Inspect Schema

```
from pyspark.sql import functions as F, Window
from pyspark.sql.types import *
df_raw = spark.read.format('delta').load('Tables/dbo/bronze_transactions')
print(f'Bronze rows: {df_raw.count():,}')
df_raw.printSchema()
```

nb_silver_transformation.ipynb — Cells 1–2: loads the raw Bronze transactions table (6,362,620 rows) and confirms every column is currently string-typed.

8.2 Rename & Cast Columns

```
df_renamed = df_raw.select(
    F.col('step').cast(IntegerType()).alias('step_number'),
    F.col('type').alias('transaction_type'),
    F.col('amount').cast(DoubleType()).alias('amount'),
    F.col('nameOrig').alias('origin_account'),
    F.col('oldbalanceOrg').cast(DoubleType()).alias('origin_balance_before'),
    F.col('newbalanceOrig').cast(DoubleType()).alias('origin_balance_after'),
    F.col('nameDest').alias('destination_account'),
    F.col('oldbalanceDest').cast(DoubleType()).alias('dest_balance_before'),
    F.col('newbalanceDest').cast(DoubleType()).alias('dest_balance_after'),
    F.col('isFraud').cast(BooleanType()).alias('is_fraud'),
    F.col('isflaggedFraud').cast(BooleanType()).alias('is_flagged_fraud'))
```

nb_silver_transformation.ipynb — Cell 3: renames every cryptic banking-system column into a clear, descriptive name and casts amounts/balances to DoubleType and fraud flags to BooleanType.

8.3 Derived Balance Metrics

```
# ── STEP 3: Calculate balance change metrics ──────────────────
df_enriched = df_renamed.withColumn('origin_balance_change',F.col('origin_balance_after') -
F.col('origin_balance_before')) \
   .withColumn('balance_discrepancy',F.abs(F.col('origin_balance_before') -
F.col('origin_balance_after') -
F.col('amount'))) \
   .withColumn('transaction_id',
       F.concat_ws('_', F.col('origin_account'),
F.col('step_number').cast(StringType())))
```

nb_silver_transformation.ipynb — Cell 5: computes origin_balance_change (how much the origin balance moved), balance_discrepancy (how far the balance movement deviates from the stated transaction amount — a core fraud-detection signal), and a synthetic transaction_id built from account + step.

8.4 Synthetic Timestamp & Calendar Fields

```
from pyspark.sql import functions as F

base_date = "2025-01-01 00:00:00"

df_enriched = (
```

```
    df_enriched
    .withColumn(
        "transaction_datetime",
        F.to_timestamp(
            F.from_unixtime(
                F.unix_timestamp(F.lit(base_date))
                + F.col("step_number") * 3600
            )
        )
    )
    .withColumn("transaction_date", F.to_date(F.col("transaction_datetime")))
    .withColumn("transaction_year", F.year(F.col("transaction_datetime")))
    .withColumn("transaction_month", F.month(F.col("transaction_datetime")))
    .withColumn("transaction_quarter", F.quarter(F.col("transaction_datetime")))
    .withColumn("transaction_hour", F.hour(F.col("transaction_datetime")))
    .withColumn("day_of_week", F.dayofweek(F.col("transaction_datetime")))
    .withColumn("is_weekend", F.dayofweek(F.col("transaction_datetime")).isin([1, 7]))
)
```

nb_silver_transformation.ipynb — Cell 7: the source step field is a sequential hour counter, not a real date, so a synthetic transaction_datetime is constructed by adding step_number hours to a fixed base date (2025-01-01). Year, month, quarter, hour, weekday, and a weekend flag are then derived from it, enabling all of the time-based Gold aggregations in Section 9.

8.5 Data Quality Flags

```
# ── STEP 4: Data quality flags ──────────────────
df_clean = df_enriched \
   .withColumn('dq_negative_amount',    F.col('amount') < 0) \
   .withColumn('dq_zero_amount',        F.col('amount') == 0) \
   .withColumn('dq_null_origin',        F.col('origin_account').isNull()) \
   .withColumn('dq_large_discrepancy',  F.col('balance_discrepancy') > 1) \
   .withColumn('is_valid_transaction',
       ~F.col('dq_negative_amount') &
       ~F.col('dq_zero_amount') &
       ~F.col('dq_null_origin'))
```

nb_silver_transformation.ipynb — Cell 9: flags each row with explicit, inspectable data-quality booleans (negative amount, zero amount, null origin account, large balance discrepancy) and rolls the first three into a single is_valid_transaction flag.

```
df_silver = df_clean.filter(F.col('is_valid_transaction') == True)
```

nb_silver_transformation.ipynb — Cell 10: derives df_silver, a filtered DataFrame containing only rows flagged as valid.

8.6 Persist to Silver

```
df_clean.write \
    .format("delta") \
    .mode("overwrite") \
    .partitionBy(
        "transaction_year",
        "transaction_month"
    ) \
    .saveAsTable("silver_transactions")
```

nb_silver_transformation.ipynb — Cell 11: writes the Silver table, partitioned by year and month for efficient time-based querying.

> *Code-review note: the write in Cell 11 persists df_clean (all 6,362,620 rows, including the 16 rows flagged dq_negative_amount / dq_zero_amount / dq_null_origin), not the filtered df_silver DataFrame created one cell earlier. The print statement in the next cell ("Silver transactions written: 6,362,604 rows") describes df_silver's count, but that is not actually what gets written to the silver_transactions table — the table itself still contains all 6,362,620*

> *rows with their dq_* flags attached. This is flagged in Section 15 as a fix: either write df_silver instead of df_clean, or update the print statement and downstream documentation to reflect that invalid rows are retained (flagged, not removed) in Silver.*

```
print(f'Silver transactions written: {df_silver.count():,} rows')
print(f'Dropped (invalid): {df_raw.count() - df_silver.count():,} rows')
display(df_silver.limit(5))
```

nb_silver_transformation.ipynb — Cell 12 output: Silver transactions written: 6,362,604 rows; Dropped (invalid): 16 rows — describing df_silver, per the note above.

8.7 Reference Data via Dataflow Gen2

Customers, products, and branches are cleansed through a Dataflow Gen2 (Power Query) rather than a notebook. The screenshot below shows this dataflow's applied steps against bronze_customers: trimming text fields, changing column types, adding conditional columns, and lower-casing text — the same categories of cleansing performed in code for transactions, expressed instead as a low-code Power Query pipeline.

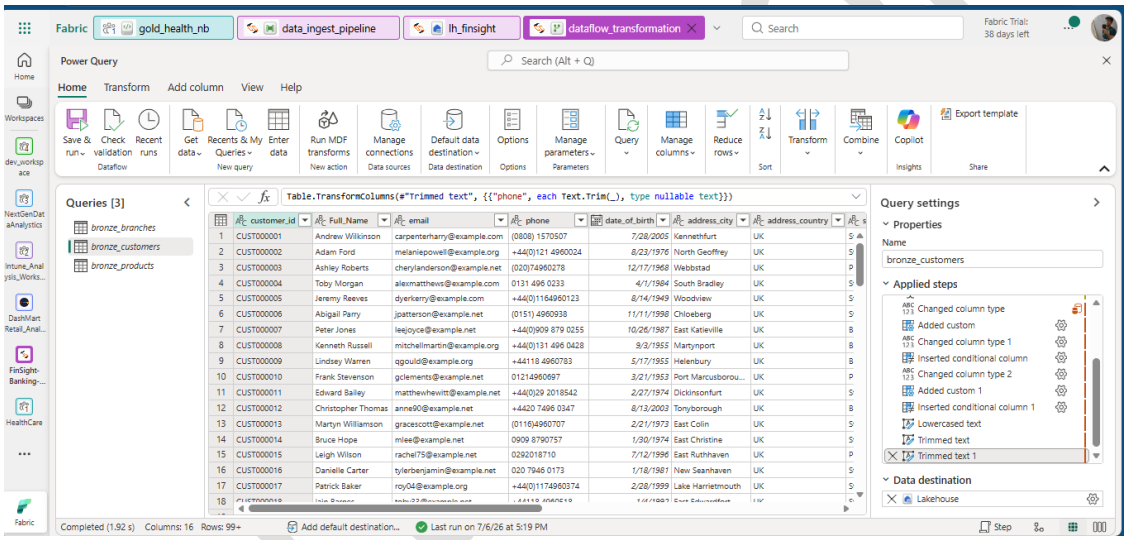

Figure 8.1 — Dataflow Gen2 (Power Query) transforming bronze_customers — trimmed text, changed column types, and conditional columns — landing the result as silver_customers.

- silver_customers: trimmed phone/address text, standardized date_of_birth typing, derived fields (e.g., age_band), lower-cased email.
- silver_products: type and null handling on fee and interest_rate.
- silver_branches: standardized region naming and is_active typing.

9. Gold Layer — Business Aggregations (nb_gold_daily_summary.ipynb)

The Gold notebook reads each Silver table once and produces four independent, purpose-built aggregation tables, each written as its own managed Delta table so Power BI can query small, pre-aggregated results instead of scanning millions of Silver rows on every report refresh.

9.1 Transaction Summary

```
from pyspark.sql import functions as F
df = spark.read.table("silver_transactions")

print(df.count())
display(df.limit(5))
```

nb_gold_daily_summary.ipynb — Cells 1–2: imports and loads the Silver transactions table (6,362,620 rows, consistent with the note in Section 8.6).

```
gold_summary = (
    df.groupBy(
        "transaction_type",
        "transaction_year",
        "transaction_month"
    )
    .agg(
        F.count("*").alias("transaction_count"),
        F.sum("amount").alias("total_amount"),
        F.avg("amount").alias("average_amount"),
        F.sum(F.col("is_fraud").cast("int")).alias("fraud_count")
    )
)
gold_summary.write \
    .format("delta") \
    .mode("overwrite") \
    .saveAsTable("gold_transaction_summary")
```

nb_gold_daily_summary.ipynb — Cells 3–5: aggregates transaction count, total and average amount, and fraud count by transaction type and calendar month, then persists to gold_transaction_summary — the source for the Executive Summary dashboard (Section 12.1).

9.2 Branch Summary

```
df_branches = spark.read.table("silver_branches")

print(df_branches.count())
display(df_branches.limit(5))
gold_branch_summary = (
    df_branches
    .groupBy("region")
    .agg(
        F.count("*").alias("total_branches"),

        F.sum(
            F.when(F.col("is_active") == True, 1).otherwise(0)
        ).alias("active_branches"),

        F.sum(
```

```
            F.when(F.col("is_active") == False, 1).otherwise(0)
        ).alias("inactive_branches"),

        F.round(F.avg("staff_count"), 1).alias("avg_staff_count"),

        F.max("staff_count").alias("max_staff_count"),

        F.min("staff_count").alias("min_staff_count")
    )
)
gold_branch_summary.write \
    .format("delta") \
    .mode("overwrite") \
    .saveAsTable("gold_branch_summary")
```

nb_gold_daily_summary.ipynb — Cells 6–8: aggregates branch counts (active/inactive) and staffing statistics by region (200 branches total), feeding the Product & Branch Analytics dashboard.

9.3 Customer Summary

```
df_customers = spark.read.table("silver_customers")
display(df_customers)
gold_customer_summary = (
    df_customers
    .groupBy("segment", "age_band")
    .agg(
        F.count("*").alias("customer_count"),
        F.round(F.avg("annual_income"),2).alias("avg_annual_income"),
        F.round(F.avg("credit_score"),2).alias("avg_credit_score"),
        F.sum(
            F.when(F.col("is_active") == True, 1).otherwise(0)
        ).alias("active_customers"),
        F.sum(
            F.when(F.col("is_active") == False, 1).otherwise(0)
        ).alias("inactive_customers")
    )
    .orderBy("segment", "age_band")
)
display(gold_customer_summary)
gold_customer_summary.write \
    .format("delta") \
    .mode("overwrite") \
    .saveAsTable("gold_customer_summary")
```

nb_gold_daily_summary.ipynb — Cells 9–11: aggregates customer count, average income and credit score, and active/inactive counts by segment and age band (50,000 customers total), feeding the Customer Analytics dashboard.

9.4 Product Summary

```
df_products=spark.read.table("silver_products")
display(df_products)
gold_product_summary = (
    df_products
    .groupBy("category")
    .agg(
        F.count("*").alias("total_products"),
        F.avg("fee").alias("avg_fee"),
        F.max("fee").alias("max_fee"),
```

```
        F.min("fee").alias("min_fee"),
        F.avg("interest_rate").alias("avg_interest_rate"),
        F.max("interest_rate").alias("max_interest_rate"),
        F.min("interest_rate").alias("min_interest_rate")
    )
    .orderBy("category")
)
display(gold_product_summary)
gold_product_summary.write \
    .format("delta") \
    .mode("overwrite") \
    .saveAsTable("gold_product_summary")
```

nb_gold_daily_summary.ipynb — Cells 12–14: aggregates fee and interest-rate ranges by product category (10 products across 6 categories), feeding the Product & Branch Analytics dashboard.

9.5 Gold Table Reference Summary

Gold Table	Grain	Key Metrics
gold_transaction_summary	transaction_type, year, month	transaction_count, total_amount, average_amount, fraud_count
gold_branch_summary	region	total_branches, active/inactive_branches, avg/max/min_staff_count
gold_customer_summary	segment, age_band	customer_count, avg_annual_income, avg_credit_score, active/inactive_customers
gold_product_summary	category	total_products, avg/max/min_fee, avg/max/min_interest_rate

10. Pipeline Orchestration

A Fabric Data Pipeline chains the four raw-file Copy Data activities, the Bronze validation notebook, the Silver processing (notebook + Dataflow Gen2), and the Gold aggregation notebook, finishing with a Power BI semantic model refresh.

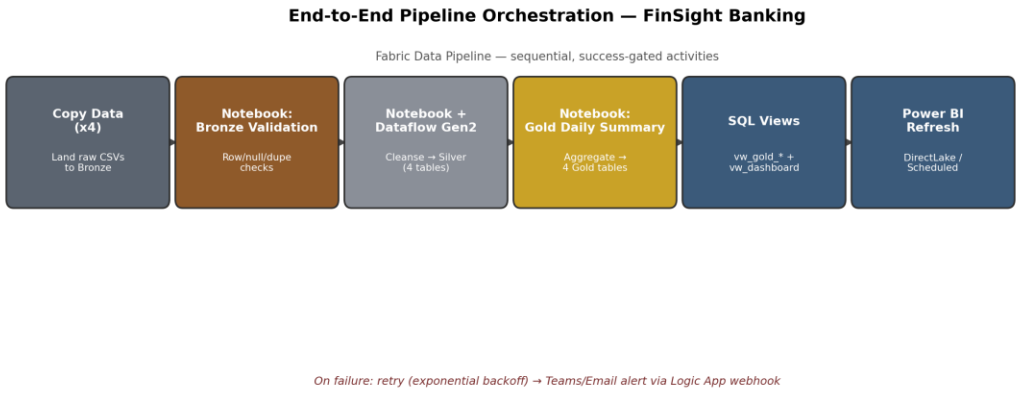

Figure 10.1 — End-to-end orchestration flow: parallel Copy Data → Bronze validation → Silver (notebook + dataflow) → Gold → SQL views → Power BI refresh.

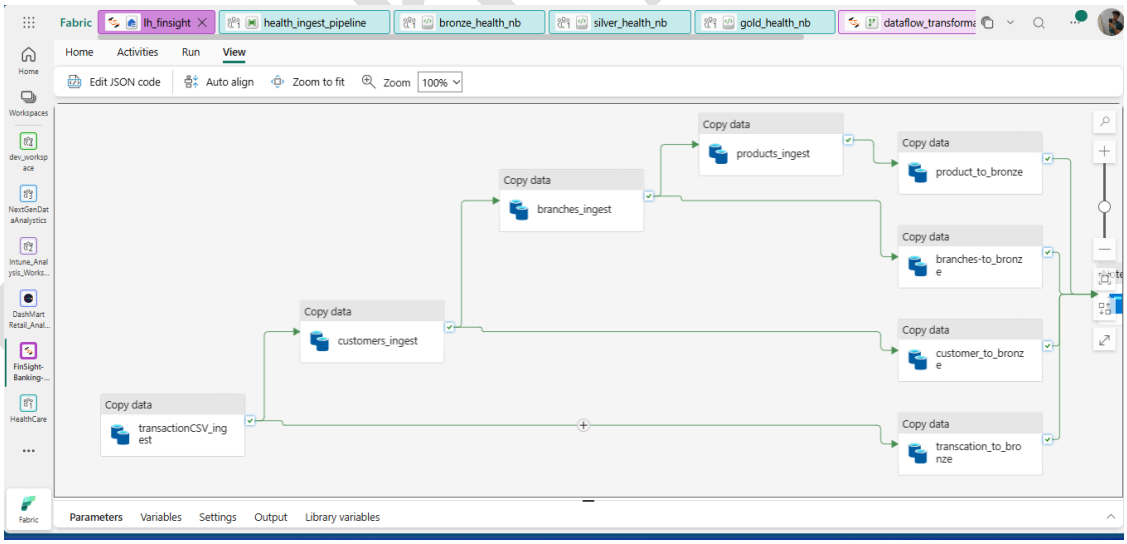

Figure 10.2 — The health_ingest_pipeline canvas: four parallel Copy Data activities (customers_ingest, branches_ingest, products_ingest, transactionCSV_ingest) landing raw files into their respective Bronze tables.

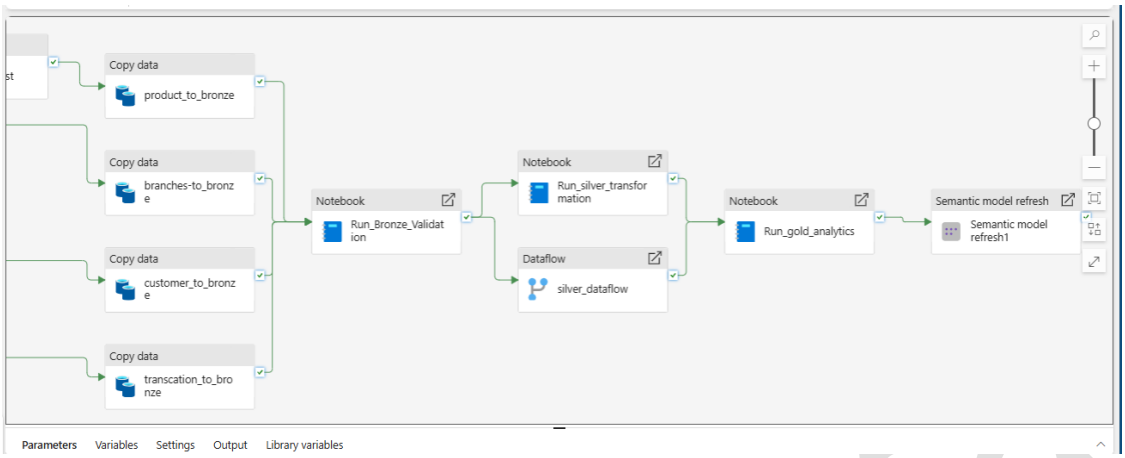

Figure 10.3 — Downstream pipeline stages: Run_Bronze_Validation notebook, the silver_dataflow (Dataflow Gen2), Run_silver_transformation and Run_gold_analytics notebooks, and a final Semantic model refresh activity.

10.1 Fault Tolerance & Monitoring

- Retry policy: each pipeline activity is configured to retry (e.g., up to three attempts with exponential backoff) to absorb transient network or storage errors.
- Alerting: failure paths trigger a webhook (Teams or email via Logic Apps) so the engineering team is notified immediately of a failed run.
- Validation gate: the Run_Bronze_Validation notebook activity runs before Silver processing begins, so a Bronze load that fails its row-count or null-key checks can halt the pipeline before bad data reaches Silver or Gold.

11. SQL Analytics Endpoint & Serving Views

The lh_finsight Lakehouse automatically exposes a read-only SQL Analytics Endpoint over its Delta tables. A thin layer of views is defined on top of the four Gold tables, giving Power BI a stable naming convention (vw_gold_*) independent of the underlying physical table names, plus a combined vw_dashboard view for report-level convenience.

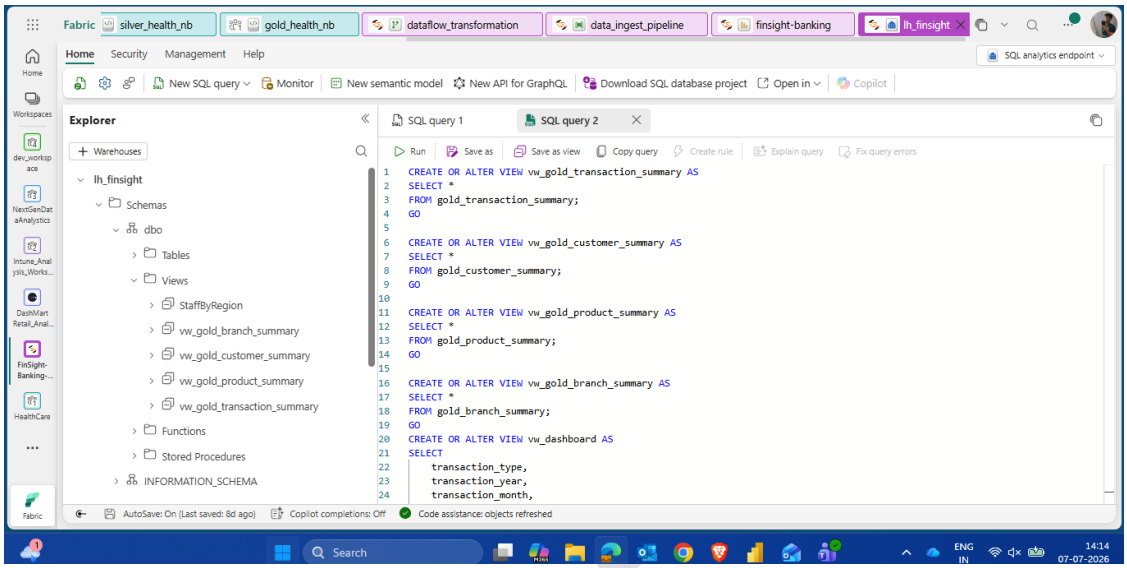

Figure 11.1 — The lh_finsight SQL Analytics Endpoint, showing CREATE OR ALTER VIEW statements for vw_gold_transaction_summary, vw_gold_customer_summary, vw_gold_product_summary, vw_gold_branch_summary, and vw_dashboard.

```
CREATE OR ALTER VIEW vw_gold_transaction_summary AS
SELECT * FROM gold_transaction_summary;
GO

CREATE OR ALTER VIEW vw_gold_customer_summary AS
SELECT * FROM gold_customer_summary;
GO

CREATE OR ALTER VIEW vw_gold_product_summary AS
SELECT * FROM gold_product_summary;
GO

CREATE OR ALTER VIEW vw_gold_branch_summary AS
SELECT * FROM gold_branch_summary;
GO

CREATE OR ALTER VIEW vw_dashboard AS
SELECT
    transaction_type,
    transaction_year,
    transaction_month,
    ...
FROM gold_transaction_summary;
GO
```

SQL view definitions as shown in Figure 11.1, mapping each Gold table to a stable vw_gold_* reporting view, plus a combined vw_dashboard view feeding the Executive Summary report page.

12. Power BI Dashboards

The FinSight Power BI report has three pages, each built on DirectLake mode against the Gold tables described in Section 9. Page navigation buttons at the bottom of each page (Executive Summary, Customer Analytics, Product & Branch Analytics) let users move between views.

12.1 Page 1 — Executive Summary

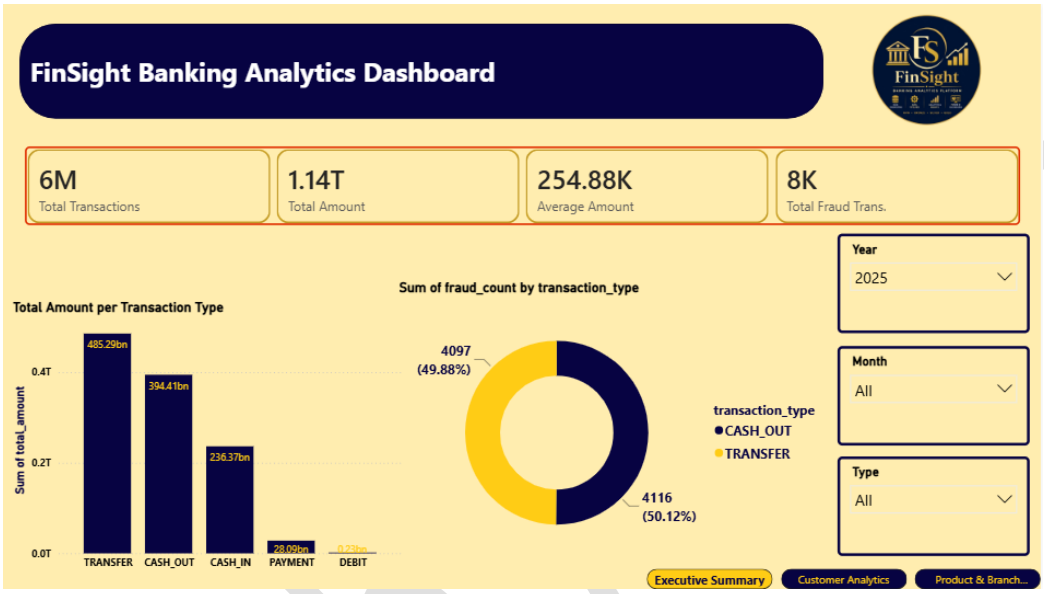

Figure 12.1 — Executive Summary: 6M total transactions, £1.14T total amount, £254.88K average amount, 8K total fraud transactions; total amount by transaction type; fraud count split by type.

This page is built from gold_transaction_summary. The KPI cards report 6 million total transactions, £1.14 trillion in total transacted amount, an average transaction amount of £254.88K, and 8,000 total fraudulent transactions. The bar chart shows total amount concentrated in TRANSFER (£485.29bn) and CASH_OUT (£394.41bn), with CASH_IN (£236.37bn) close behind and PAYMENT/DEBIT far smaller. The donut chart shows fraud is split almost evenly between CASH_OUT (4,116 cases, 50.12%) and TRANSFER (4,097 cases, 49.88%) — consistent with known fraud patterns in this type of transaction dataset, where fraud is concentrated in exactly these two transaction types.

12.2 Page 2 — Customer Analytics

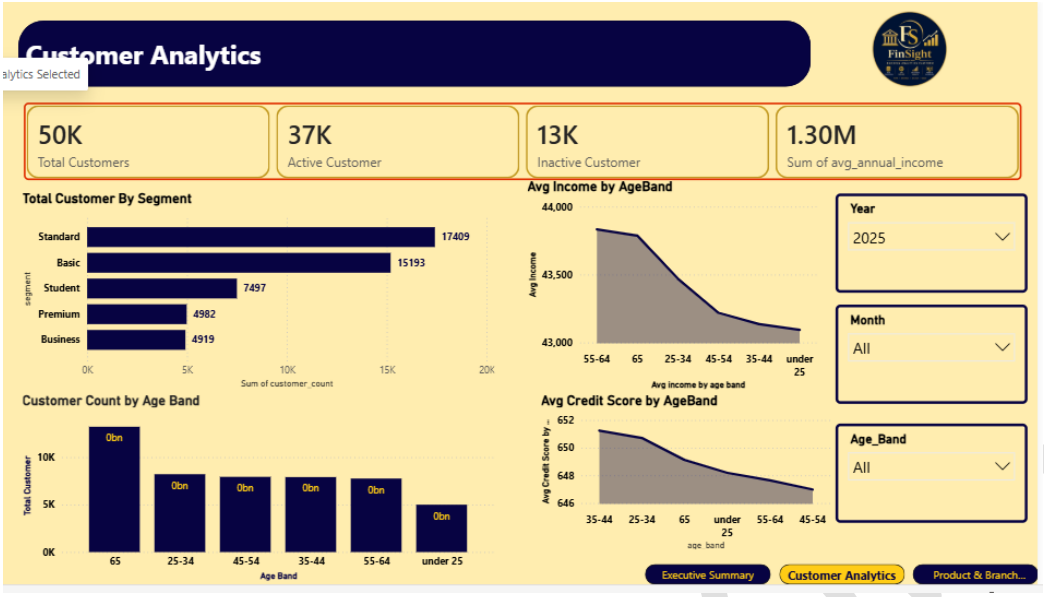

Figure 12.2 — Customer Analytics: 50K total customers, 37K active, 13K inactive, £1.30M summed average annual income; customer count and segment breakdown; average income and credit score by age band.

Built from gold_customer_summary. Of 50,000 total customers, 37,000 are active and 13,000 inactive. The segment breakdown shows Standard (17,409) and Basic (15,193) as the two largest tiers, followed by Student (7,497), Premium (4,982), and Business (4,919). The two trend charts show average income and average credit score both declining somewhat for older age bands in this sample, which is worth validating against source data assumptions rather than taken as a general banking truth.

12.3 Page 3 — Product & Branch Analytics

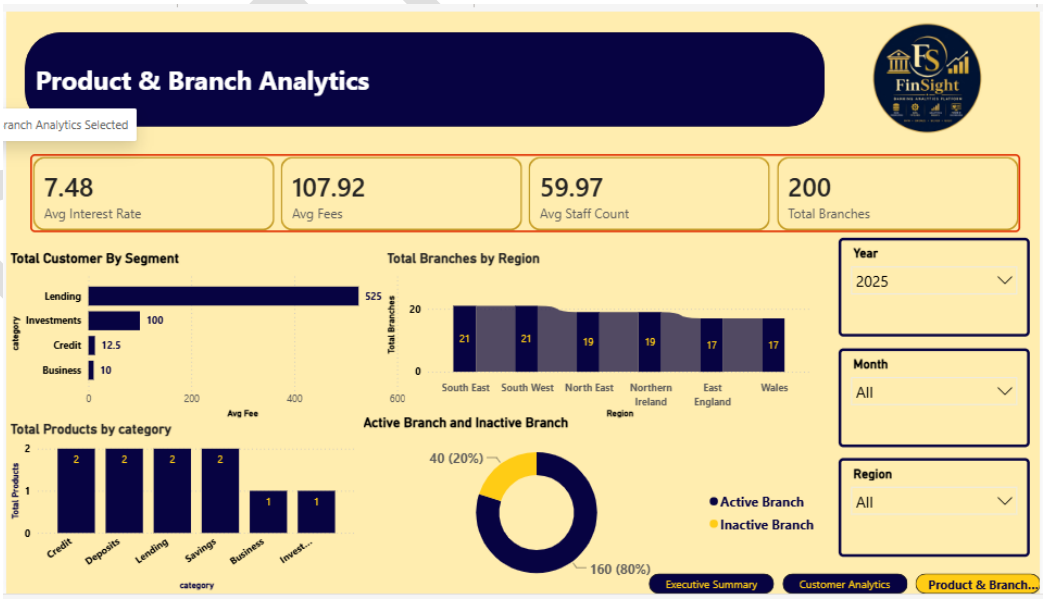

Figure 12.3 — Product & Branch Analytics: 7.48 average interest rate, £107.92 average fees, 59.97 average staff count, 200 total branches; fee-by-category chart; branches by region; products by category; active vs inactive branch split.

Built from gold_branch_summary and gold_product_summary. Across 200 branches, 160 are active (80%) and 40 inactive (20%), with an average of roughly 60 staff per branch. Branches are fairly evenly spread across six regions (South East and South West each with 21, North East and Northern Ireland each with 19, East England and Wales each with 17). The product side shows 10 products spread across 6 categories (Credit, Deposits, Lending, Savings, Business, Investments), each with 1–2 products, with Lending products carrying the highest average fee.

> *Presentation note: the bar chart titled "Total Customer By Segment" on this page actually plots Avg Fee by category (Lending, Investments, Credit, Business), not a customer segment count — the chart title appears to have been copied from the Customer Analytics page (Figure 12.2) without being updated. This is a cosmetic labeling issue to fix in the Power BI report rather than a data problem, and is listed again in Section 15.*

13. Lakehouse Optimization & Governance

13.1 Delta Lake Performance Tuning

With 6.3 million transaction rows partitioned by year and month, keeping file layout efficient matters for both cost and query latency:

```
OPTIMIZE silver_transactions;
```

Compacts small Parquet files within each year/month partition into larger, more efficient blocks, reducing file-scan overhead on read.

```
OPTIMIZE gold_transaction_summary ZORDER BY (transaction_type, transaction_year);
```

Z-Ordering co-locates rows by the two dimensions most Power BI visuals filter on — transaction type and year.

```
VACUUM silver_transactions RETAIN 168 HOURS;
```

Removes stale historical file versions beyond the retention window (168 hours = 7 days), preventing unbounded storage growth from Delta's time-travel history.

13.2 Access Control & Data Protection

- Role-based access control (RBAC): data engineers retain read/write access across Bronze, Silver, and Gold; BI analysts and branch/risk teams are restricted to read-only access on Gold tables and the Power BI semantic model.
- Sensitive-field protection: origin_account and destination_account identifiers, along with customer PII in silver_customers, are candidates for dynamic masking or restriction in the serving layer ahead of broader report distribution.

14. Testing & Data Validation

Unlike a purely illustrative testing section, FinSight already implements a concrete, automated validation step: the _nb_bronze_validation notebook (Section 7) runs a row-count, null-key, and duplicate-key check against every Bronze table and logs the result, with timestamp, to bronze_validation_log on every run. This gives the pipeline a durable, queryable audit trail of every ingestion's data-quality outcome, rather than relying on someone remembering to eyeball a display() output.

- Row-count thresholds catch a badly truncated or empty source file before it reaches Silver (e.g., bronze_transactions must have at least 1,000,000 rows).
- Null-key checks catch broken key generation upstream (e.g., a customer or product missing its identifier).
- Duplicate-key counts are recorded for visibility even where duplication is expected (see Section 7.4), giving analysts a signal without a false failure.
- In Silver, explicit dq_* boolean flags (Section 8.5) make every quality issue inspectable at the row level, rather than silently dropped.

A natural next step is to extend this same validation pattern to the Silver and Gold layers — for example, asserting that gold_transaction_summary's total transaction_count reconciles exactly against silver_transactions row counts.

15. Known Issues & Lessons Learned

15.1 silver_transactions Writes Unfiltered Data

As documented in Section 8.6, nb_silver_transformation.ipynb computes a filtered df_silver DataFrame but writes the unfiltered df_clean DataFrame to the silver_transactions table. The print statement immediately after ("Silver transactions written: 6,362,604 rows") is therefore inaccurate — the table actually holds all 6,362,620 rows, with the 16 invalid rows retained and flagged via is_valid_transaction = false rather than removed. Because nb_gold_daily_summary.ipynb reads directly from silver_transactions without filtering on is_valid_transaction, the 16 flagged rows also flow into gold_transaction_summary. The practical impact at this row count is negligible (16 rows out of 6.36 million), but the fix is straightforward: either write df_silver instead of df_clean, or add an explicit .filter(F.col('is_valid_transaction')) in the Gold notebook before aggregating.

15.2 Validation key_col Naming

As noted in Section 7.4, the key_col label in the validation configuration implies uniqueness, but for bronze_transactions it is expected to repeat (an account has many transactions). Renaming this configuration field (e.g., to reference_col) would make the validation notebook's intent clearer to future maintainers.

15.3 Power BI Chart Title Mismatch

As noted in Section 12.3, the Product & Branch Analytics page has a bar chart labeled "Total Customer By Segment" that actually displays average fee by product category — a leftover title from the Customer Analytics page. This should be corrected before the report is presented externally.

15.4 General Takeaways

- Building an explicit, automated validation notebook ahead of Silver processing (rather than relying on ad hoc display() checks) proved valuable — it gave the pipeline a queryable, timestamped audit trail (bronze_validation_log) almost for free.
- Retaining data-quality flags as columns (dq_negative_amount, dq_zero_amount, etc.) rather than silently dropping rows makes issues like the one in Section 15.1 possible to catch by inspection, which is how it was found here.

16. Future Enhancements

- Fix the Silver write to persist df_silver (or filter Gold on is_valid_transaction) so invalid transaction rows are consistently excluded downstream.
- Rename the validation key_col configuration field to better reflect that duplication is expected for transaction-level reference columns.
- Correct the mislabeled chart title on the Product & Branch Analytics Power BI page.
- Extend the bronze_validation_log pattern to Silver and Gold, with reconciliation checks between layers (e.g., row-count parity, fraud-count parity).
- Apply dynamic data masking to account identifiers and customer PII fields ahead of wider report distribution.
- Add scheduled OPTIMIZE / ZORDER / VACUUM maintenance as pipeline activities rather than ad hoc manual commands.

17. Conclusion

FinSight delivers a working, validated Medallion pipeline that turns raw banking CSV extracts — transactions, customers, products, and branches — into four governed Gold tables and a three-page executive Power BI report, running on Microsoft Fabric with Delta Lake as the storage format. The Bronze validation gate, the PySpark Silver transformation for transactions, the Dataflow Gen2 cleansing for reference data, and the Gold aggregation notebook documented in this report together provide a solid, auditable foundation for fraud monitoring, customer segmentation, and branch/product performance reporting — along with a clear, prioritized list of fixes (Section 16) to close the small gaps identified during this review.